



Construir um mapa de métricas dos indicadores de desempenho de uma empresa

Agendar mentoria

Algoritmos e estrutura de dados

Entenda seu projeto



Fase 1

Período de entrega
De: 25/05/2026 - 00:00h
Até: 01/06/2026 - 23:59h



Fase 2

Período de entrega
De: 29/06/2026 - 00:00h
Até: 06/07/2026 - 23:59h



Fase 3

Período de entrega
De: 27/08/2026 - 00:00h
Até: 02/09/2026 - 23:59h

Entrega Final

Período de entrega
De: 16/09/2026 - 00:00h
Até: 22/09/2026 - 23:59h

Atividade Prática 1: Gerenciando Coleções de Dados com Listas, Tuplas e Dicionários

Objetivos:

- Aplicar o uso de listas para armazenar e manipular coleções de dados dinâmicos;
- Utilizar tuplas para representar dados que não devem ser alterados (imutáveis);
- Empregar dicionários para mapear informações usando o conceito de chave-valor;
- Praticar a combinação dessas estruturas para modelar cenários do mundo real;
- Desenvolver funções que operam sobre essas estruturas de dados.

Instruções

Parte 1: Catálogo de Produtos da Loja (Listas e Tuplas)

Imagine que você está desenvolvendo um sistema para uma pequena loja online. Você precisa catalogar os produtos. Cada produto terá um código (string), nome (string) e preço (float). Uma vez cadastrado, o código e o nome de um produto não devem mudar, mas o preço pode ser atualizado.

1. Representando um Produto:

- Decida qual estrutura (ou combinação) é mais adequada para representar as informações fixas de um produto (código e nome). Justifique sua escolha considerando a imutabilidade.
- Como você armazenaria o preço de forma que ele possa ser atualizado?

2. Criando o Catálogo:

- Crie uma lista chamada `catalogo_loja` para armazenar vários produtos;
- Adicione pelo menos 4 produtos ao seu catálogo. Cada item na lista `catalogo_loja` deve conter o código, nome e preço do produto.

- Exemplo de um produto: Código "LIV001", Nome "O Senhor dos Anéis", Preço 79,90.

3. Funções de Gerenciamento do Catálogo:

- Crie uma função `adicionar_produto_catalogo(catalogo, codigo, nome, preco)` que adiciona um novo produto ao catálogo;
- Crie uma função `buscar_produto_por_codigo_catalogo(catalogo, codigo_busca)` que retorna os dados do produto (ou `None` se não encontrado);
- Crie uma função `listar_todos_produtos(catalogo)` que imprima de forma organizada todos os produtos do catálogo (código, nome, preço).

4. Testes: Chame suas funções para adicionar um novo produto e listar todos os produtos. Tente buscar um produto existente e um inexistente.

Parte 2: Carrinho de Compras (Listas e Dicionários)

Agora, vamos simular um carrinho de compras. O carrinho deve armazenar os produtos que um cliente deseja comprar e a quantidade de cada um.

1. Representando Itens no Carrinho:

- Crie uma lista chamada `carrinho_compras`;
- Cada item no `carrinho_compras` será um dicionário representando um produto adicionado, contendo as chaves: `"codigo_produto"` (string) e `"quantidade"` (inteiro).

2. Funções do Carrinho:

Crie uma função `adicionar_ao_carrinho(carrinho, catalogo, codigo_produto_adicionar, quantidade_adicionar)`:

Esta função deve primeiro verificar se o `codigo_produto_adicionar` existe no `catalogo_loja` (você pode reutilizar a função da Parte 1).

Se o produto existir:

- Verifique se o produto já está no carrinho;
- Se já estiver, apenas some a `quantidade_adicionar` à quantidade existente;
- Se não estiver, adicione um novo dicionário ao carrinho com o código do produto e a quantidade;
- Se o produto não existir no catálogo, imprima uma mensagem de erro.

Crie uma função `calcular_total_carrinho(carrinho, catalogo)`:

- Esta função deve percorrer os itens no carrinho;
- Para cada item, buscar o preço do produto no `catalogo_loja`;
- Calcular o subtotal (`preço * quantidade`) para cada item e somar tudo para obter o valor total do carrinho;
- Retornar o valor total;
- Crie uma função `visualizar_carrinho(carrinho, catalogo)` que imprima de forma organizada os itens do carrinho, mostrando o nome do produto (buscando no catálogo), a quantidade e o subtotal de cada item.

3. Testes:

- Adicione alguns produtos ao catálogo (se ainda não o fez);
- Use adicionar_ao_carrinho para adicionar diferentes produtos e quantidades. Tente adicionar o mesmo produto mais de uma vez. Tente adicionar um produto que não existe no catálogo;
- Use visualizar_carrinho para ver o conteúdo;
- Use calcular_total_carrinho para obter o valor final.

Parte 3: Dados de Clientes (Dicionários e Tuplas)

Vamos armazenar informações básicas de clientes. Para cada cliente, queremos guardar um ID único (inteiro), nome (string) e um histórico de coordenadas de suas últimas localizações de acesso (representadas como tuplas (latitude, longitude)).

1. Estrutura de Dados dos Clientes:

- Crie um dicionário chamado dados_clientes;
- As chaves deste dicionário serão os IDs dos clientes.

Os valores serão outros dicionários, cada um contendo:

- "nome": (string);
- "historico_loc": uma lista de tuplas, onde cada tupla é (latitude, longitude).

2. Funções de Gerenciamento de Clientes:

- Crie uma função registrar_cliente(clientes, id_cliente, nome_cliente) que adiciona um novo cliente ao dicionário dados_clientes (com um histórico de localizações vazio inicialmente). Verifique se o ID já existe;
- Crie uma função adicionar_localizacao_cliente(clientes, id_cliente, latitude, longitude) que adiciona uma nova tupla de coordenada ao historico_loc de um cliente específico. Verifique se o cliente existe;
- Crie uma função obter_ultima_localizacao(clientes, id_cliente) que retorna a última tupla de localização registrada para um cliente (ou None se não houver histórico ou o cliente não existir).

3. Testes:

- Registre alguns clients;
- Adicione algumas localizações para eles;
- Obtenha a última localização de um cliente.

Critérios de Avaliação

- Escolha e uso apropriado de listas, tuplas e dicionários para cada cenário;
- Lógica correta e funcionalidade das funções implementadas;
- Clareza do código e uso de nomes descritivos para variáveis e funções;
- Implementação de verificações básicas (ex: produto existe no catálogo, ID de cliente já cadastrado);
- Testes que demonstrem o funcionamento das diferentes partes do sistema.

Entrega

- Um único arquivo Python (.py) contendo todo o código das Partes 1, 2 e 3, incluindo as chamadas de teste para cada parte, com os resultados impressos de forma organizada;

- Adicione comentários no seu código para explicar as decisões de design das estruturas de dados e a lógica das funções.



Modelo de Entrega -
Atividade Prática

Título

0.17MB

Parabéns você chegou ao
fim desta atividade!

Atividade anterior

Próxima atividade

Tem alguma dúvida? Clique aqui e fale com o professor.